什么是交叉熵

PyTorch 理论精读 · 损失函数专题
整理于 2026-10-09

一句话版本

交叉熵 = 用"错误的猜测"去编码真相时，平均要多付多少代价。
在分类任务里，它简化为一句话：**取模型给正确类别的概率 p，损失就是 -log(p)**。

从亲手实验说起

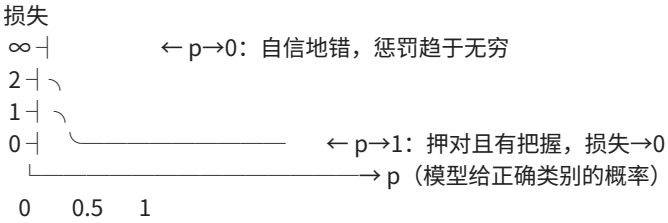
在 practice04 任务 2 中，logits 是 [2.0, 1.0, 0.1]，softmax 后概率是 [0.66, 0.24, 0.10]：

```
正确标签 = 0：loss = -log(0.66) = 0.42  ← 押对了，罚得少
正确标签 = 2：loss = -log(0.10) = 2.32  ← 押错了，罚得多
```

交叉熵只关心一件事：**你给正确答案分了多少概率**。其他类别的概率怎么分配，它不管（它们通过 softmax 的分母间接参与）。

为什么用 -log(p) 而不是 1-p?

两个函数都满足"概率越高损失越低"，但形状天差地别。看 -log(p) 的曲线：



三个关键性质：

1. **p=0.5 时损失约 0.69**——半信半疑，温和地罚
2. **p→0 时损失爆炸**——模型说"绝不可能是猫"结果真是猫，重罚。梯度也随之巨大，模型被猛拉回来
3. **p→1 时损失平缓**——已经押对了，边际改进收益小，梯度温柔

如果用 1-p，p=0.01（自信地错）只罚 0.99，和 p=0.4（不太确定）罚 0.6 差别不大——**模型犯了最严重的错误却没被狠狠教训**。这就是为什么分类任务几乎不用别的损失。

再往深一层：交叉熵的信息论出身

"熵"来自信息论，衡量一个分布的不确定性。交叉熵的完整定义是：

$$H(p, q) = -\Sigma\, p(i) \cdot \log(q(i))$$

p = 真实分布（正确答案是 one-hot：只有正确类是 1，其余是 0）
q = 模型预测的概率分布

因为 p 只有正确类别那一项是 1，求和里其他项全是 0——**所以完整的交叉熵公式在分类任务里退化成了 -log(正确类的概率)**。手写的 loss = -prob[label].log() 就是在算完整公式，只是 0 项不用写。

直觉理解：log(q(i)) 是"用模型的猜测 q 去编码类别 i 需要的比特数"，乘以真实概率 p(i)
求和，就是"平均编码代价"。模型猜得越准（q 接近 p），代价越接近理论最小值（真实分布的熵）。

为什么它求导特别干净

交叉熵和 softmax 是天生一对。softmax 输出 → 交叉熵损失，对 logits 求导后的梯度是：

梯度 = q - p　（预测概率 − 真实标签）

就这么简单。预测 0.66、真实 1.0，梯度就是 -0.34，方向明确、大小适中。没有复杂的链式项相乘，数值稳定、计算便宜——这也是它统治分类任务几十年的工程原因。

串回知识体系

模型输出 logits（原始分数，可正可负）
↓ softmax：压成概率，加起来=1
↓ -log(正确类的概率)：交叉熵
↓ backward：梯度 = q - p，干净地往回传

所以那条铁律——**CrossEntropyLoss 内部已含 softmax，别在模型最后手动加**——现在能从原理上理解了：加两次 softmax，概率被压平两次，梯度 q-p 的漂亮性质就被破坏了。